async / await

Promises that read like normal code

	async	await
Where	before a function	inside async code
Timing	sync till first await	resumes as microtask
Result	always a promise	the unwrapped value
On error	promise rejects	throws, so catch it

1 One by one vs together

```
// one by one: time = a + b
await a();
await b();

// together: time = slower one
await Promise.all([a(), b()]);
```

2 try / catch / finally

```
try {
  await load();   // may reject
} catch (e) {
  show(e);   // on reject
} finally { done(); }  // always
```

TRAP What does this log?

```
[1, 2].forEach(async n => {
  await null; console.log(n);
});
console.log("done");
```

`done 1 2`

forEach does not wait. Use **for...of** or **Promise.all**.

KEY TAKEAWAY

- **async** — Always returns a promise
- **await** — Pauses only this function
- **forEach** — Does not wait for async

? Follow-up questions to expect

- **When does the code after await run?** → As a microtask, after the sync code.
- **How do you handle errors with await?** → try / catch around it.
- **Does `return p` inside try reach the catch?** → No. Write return await p.
- **Why not `forEach` with await?** → forEach ignores the promises. Use for...of.